AXIOM

Prototype Implementation Playbook

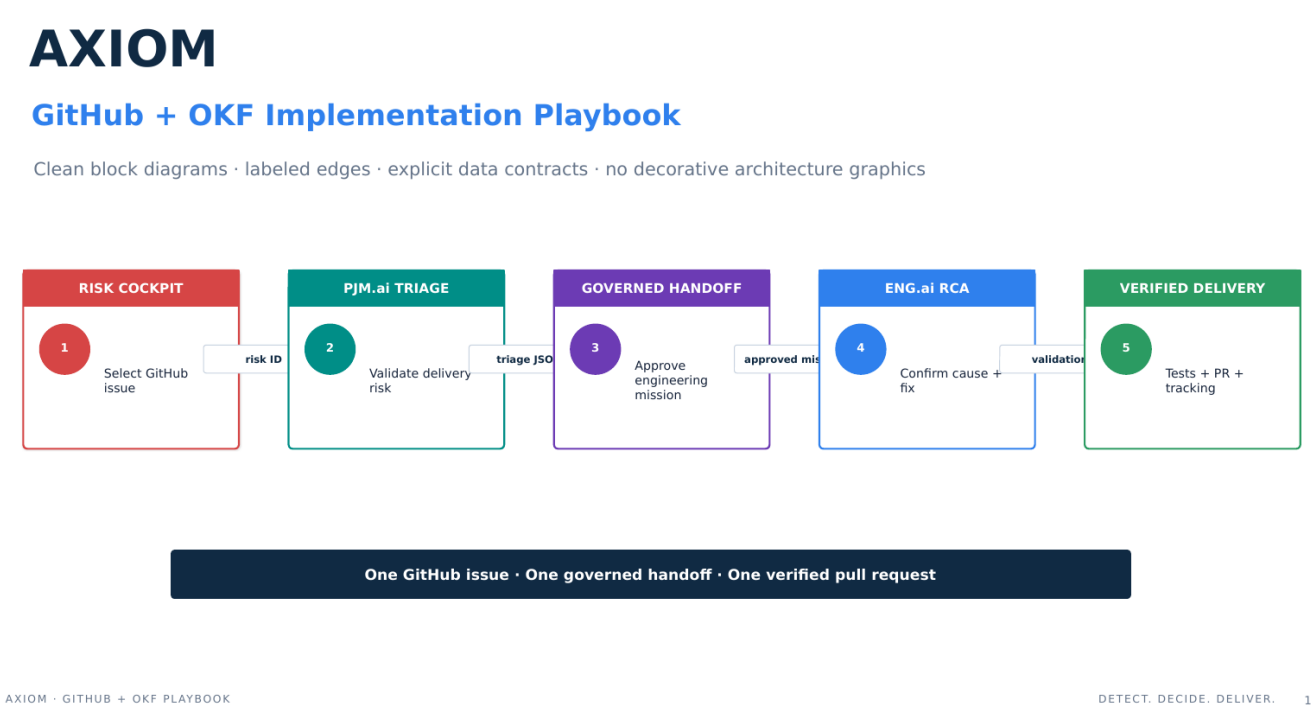
Synthetic project readiness · Google-native architecture · End-to-end risk-to-fix demo

AXIOM vertical slice: risk → governed decision → engineering resolution

Build the narrowest prototype that proves the broadest AXIOM intelligence promise.

Version 1.0 · Prepared for Google Cloud AI Builder Cup 2026

Executive decision



Executive decision: editable block-diagram view with named blocks and labeled flows.

AXIOM should be demonstrated end to end, but through one bounded defect-risk journey. PJM.ai and ENG.ai remain separate decision domains connected by a governed handoff. The winning submission is not a broad architecture showcase; it is a repeatable proof that one risk can move from fragmented project evidence to a verified engineering change.

One risk · One project · One governed handoff · One verified fix

The three-step approach

Step	Objective	Exit output
1 · Synthetic project readiness	Correct and extend the E-Key 2W evidence package so that it behaves like a real delivery project.	Versioned runtime corpus + physically separate private evaluation truth
2 · AXIOM architecture transfor	Move the vertical slice to Google Cloud while preserving PJM.ai/ENG.ai boundaries, evidence, approvals and auditability.	Deployed, observable and governed Google-native AXIOM vertical slice

mation		
3 · Golden demo execution	Run GH-ISSUE-107 from risk cockpit through triage, handoff, RCA, patch, tests and PR proof.	Ten consecutive successful runs and a three-minute submission story

Winning vertical slice

RISK COCKPIT
→ PJM.ai TRIAGE
→ PM APPROVAL
→ STRUCTURED GITHUB ISSUE HANDOFF
→ ENG.ai RCA
→ ENGINEER APPROVAL
→ TARGETED PATCH + TESTS
→ PULL REQUEST / MERGE PROOF

Playbook map

Section	What this section gives the team
1. Synthetic project readiness	What is already excellent, golden scenario, deliberate conflict, required corrections, canonical model, GitHub tracker/code additions, private truth and exit gate.
2. AXIOM architecture transformation	Current-to-target mapping, high-level and detailed architecture, decision boundaries, orchestration choice, service map, ingestion, contracts, governance, evaluation, deployment and exit gate.
3. Golden demo execution	Frozen inputs, cockpit behavior, PJM.ai output, handoff contract, ENG.ai RCA, targeted fix, tests, expected JSON, negative cases, storyboard and submission checklist.

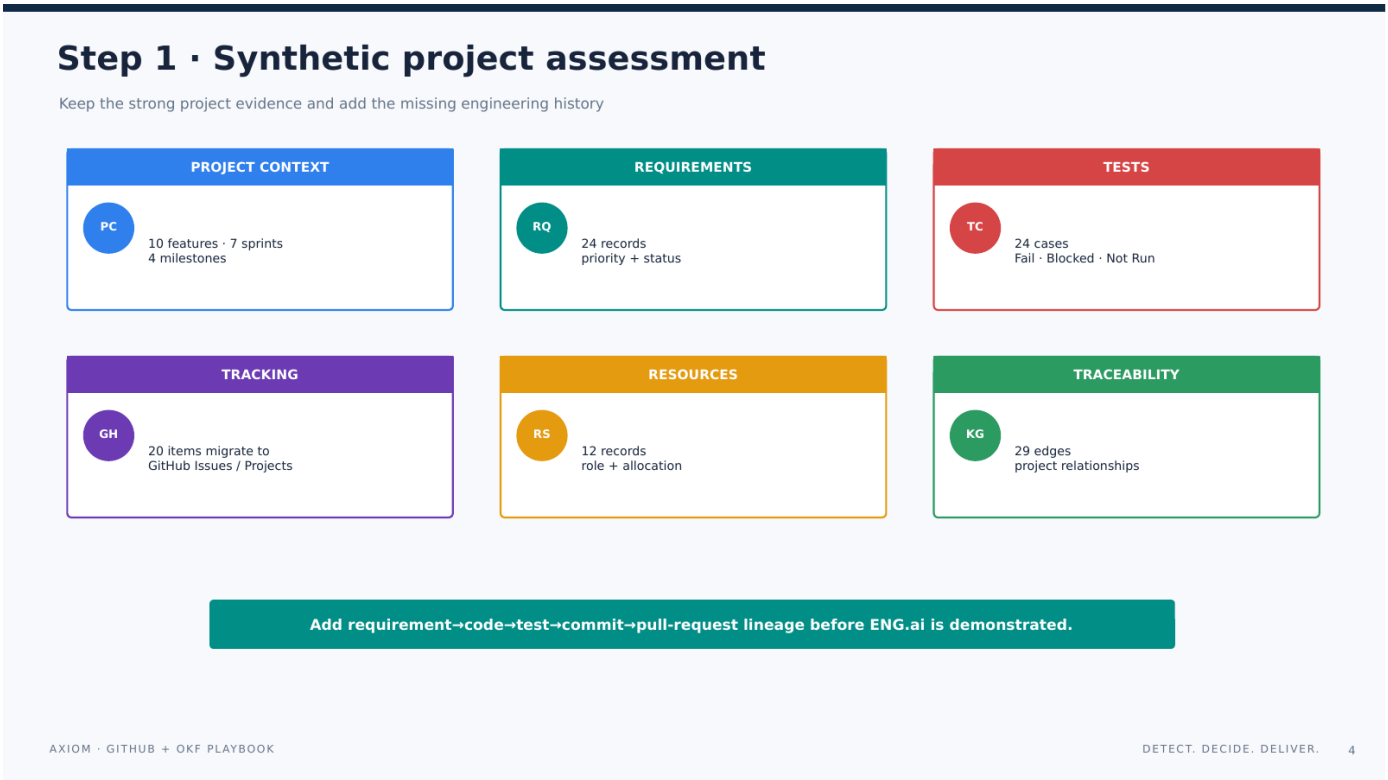
Architecture principle

PJM.ai owns delivery decisions. ENG.ai owns technical truth. A.E.G.I.S. governs actions. K.A.R.M.A. carries evidence. O.L.I. measures every run.

Non-negotiable rules

- No runtime retrieval of risk_scenarios.md or expected answers.
- No confirmed code-level root cause in the PJM.ai phase.
- No GitHub tracker or code mutation without the correct human approval.
- No “fixed” claim unless the patch applies and targeted tests pass.
- Every significant claim carries evidence IDs, source versions and confidence.

Step 1 · Synthetic project readiness



Step 1 · Synthetic project readiness: editable block-diagram view with named blocks and labeled flows.

Objective: make the E-Key 2W package internally coherent, sufficiently realistic for project and engineering intelligence, safe for public release, and measurable through private ground truth.

Synthetic project readiness: connected evidence corpus, engineering-history gaps and private evaluation separation

1.1 What is already excellent

Strength	Why it matters

Real delivery context	The package defines a fictional digital-key product, seven sprints, four milestones and delivery assumptions rather than a collection of disconnected tickets.
Cross-domain coverage	Requirements, tests, GitHub Issues, resources, plans and traceability allow AXIOM to connect technical signals to delivery impact.
Useful negative states	Fail, Blocked, Not Run, open defects and capacity constraints provide uncertainty and incomplete evidence.
Seeded intelligence patterns	Passive-access milestone risk, security/revocation risk and traceability gaps support risk, RCA and missing-evidence behaviors.
Public-friendly scope	The package states that it is synthetic and excludes actual Bosch implementation details and production cryptography.
Visible business outcome	The data can show why a defect matters to a milestone rather than merely summarizing the defect.

Source note: project_manifest.json, project_description.md, project_plan.md, risk_scenarios.md, README.md, test_cases.xlsx.

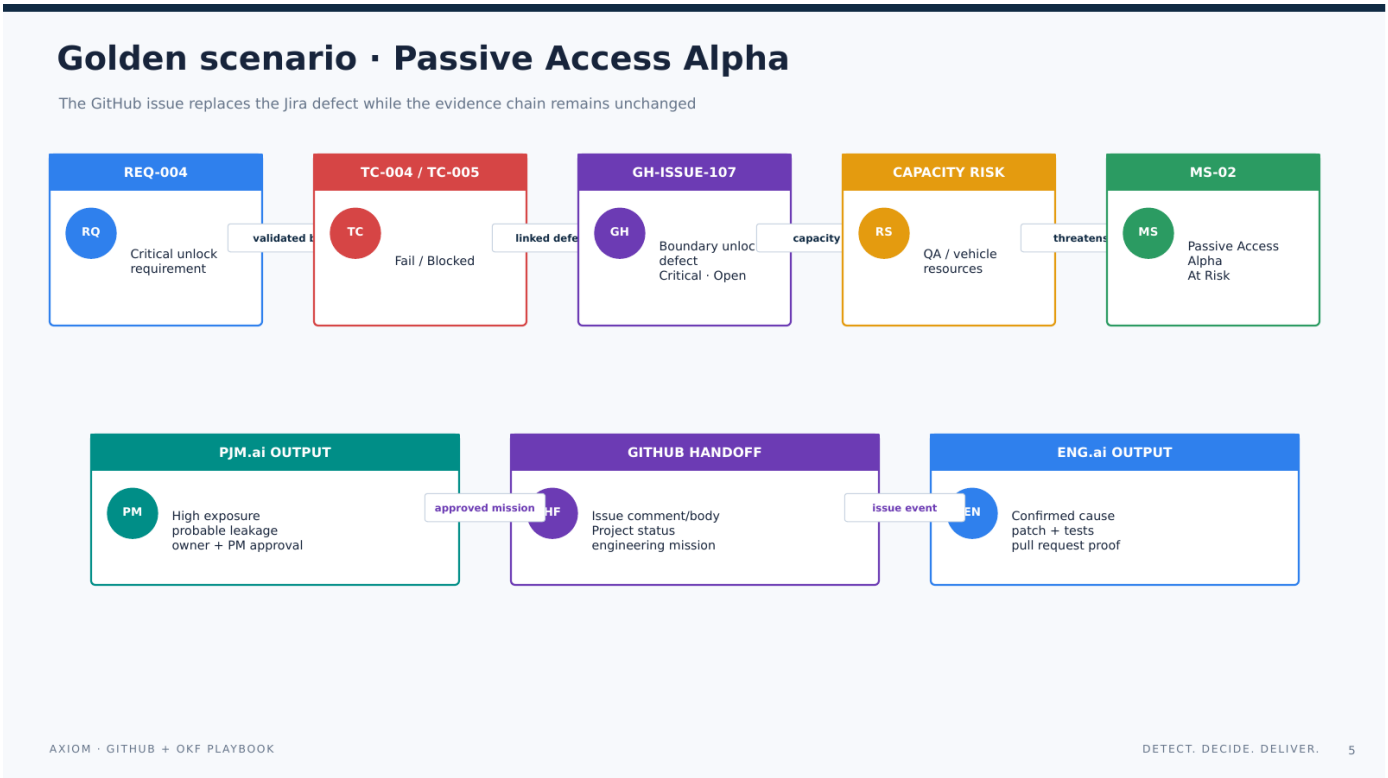
1.2 Current package inventory

Artifact	Current content	Readiness
Project manifest	10 features, 24 requirements, 24 tests, 12 resources, 7 sprints, 4 milestones, 20 GitHub Issues, 29 trace edges.	Strong
Requirements	Feature hierarchy, priority, status, component and sprint.	Strong
Tests	Requirement link, objective, result and QA owner.	Strong; owner mapping needed
GitHub	Epics, stories, tasks, bugs and risks with sprint/milestone context.	Good snapshot; lifecycle history needed
Resources	Role, domain, planned allocation, availability, sprint and milestone sheets.	Good; effective capacity and competency needed
Traceability	Project relationships such as validated_by, failed_due_to, assigned_to, threatens.	Useful; code/commit/PR links absent
Code repository	Small synthetic Python skeleton and two tests.	Insufficient for convincing RCA and patch proof
Risk scenarios	Three expected reasoning patterns.	Useful only as private evaluation truth

1.3 Readiness baseline

Dimension	Assessment	Reason
Domain realism	9/10	The project and evidence domains are credible for the selected story.
Cross-domain coverage	9/10	Enough artifact types exist to demonstrate evidence correlation.
Traceability	7/10	Project links exist; engineering lineage does not.
RCA readiness	6/10	Code and test history is too small for a strong technical investigation.
Public readiness	7/10	The package is synthetic, but metadata and repository history still require scrubbing.

1.4 Best hackathon golden scenario



1.4 Best hackathon golden scenario: editable block-diagram view with named blocks and labeled flows.

Selected scenario: Passive Access Alpha

REQ-004 Critical / In Progress
→ TC-004 Fail

→ TC-005 Blocked
→ GH-ISSUE-107 Critical / Open

REQ-006 Performance / In Progress
→ TC-007 Fail
→ GH-ISSUE-108 High / Open

GH-ISSUE-119 QA capacity risk + vehicle resources heavily allocated
→ MS-02 Passive Access Alpha = At Risk

Why this is better than the revocation scenario for the main demo

- Combines functional failure, performance failure and blocked validation.
- Contains a critical customer-relevant defect and an at-risk milestone.
- Includes requirement, test, defect, resource, sprint and milestone evidence.
- Maps naturally to the intentionally simplistic proximity-controller implementation.
- Supports a small, understandable targeted fix and regression test.

Expected PJM.ai conclusion

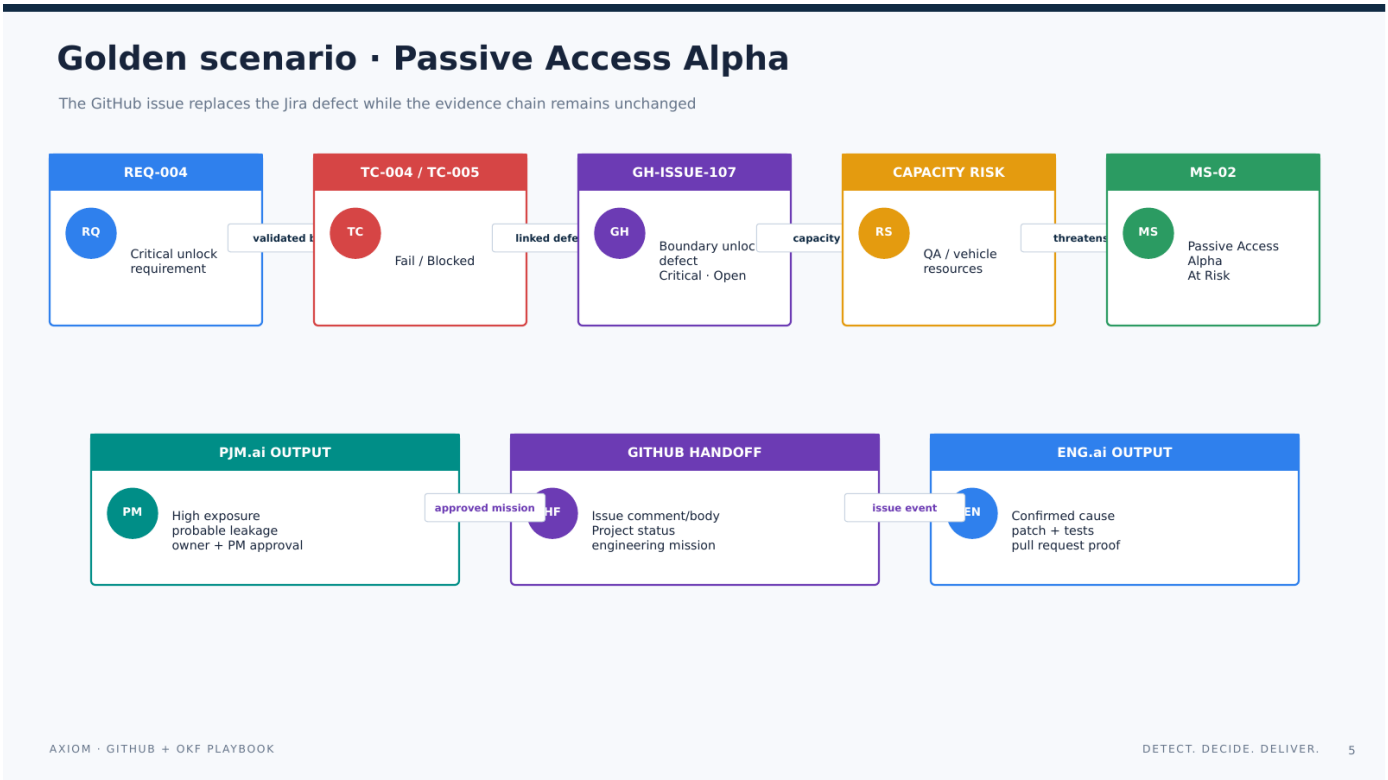
- Risk is valid and delivery exposure is high.
- Probable leakage lies across test validation and vehicle-software behavior.
- The evidence contains a test-link conflict that reduces confidence.
- Vehicle software and QA ownership is required.
- PM approval is required before posting and assigning the engineering mission.

Expected ENG.ai conclusion

- Exact affected file and symbol are identified.
- The technical cause is confirmed against requirement, code and failing test.
- A minimal patch and targeted boundary tests are proposed.
- The patch is applied only after engineer approval.
- Tests and PR metadata demonstrate verified delivery.

Source note: requirements.xlsx, test_cases.xlsx, github_issues.csv, resources.xlsx, traceability.csv, risk_scenarios.md, proximity_controller.py.

1.5 Preserve this consistency issue deliberately



1.5 Preserve this consistency issue deliberately: editable block-diagram view with named blocks and labeled flows.

Do not silently repair the GH-ISSUE-107 test-link inconsistency. Make AXIOM discover and explain it.

Evidence source	Observed statement
traceability.csv	TC-004 → GH-ISSUE-107 using failed_due_to.
github_issues.csv	GH-ISSUE-107 description says Linked to TC-005.
test_cases.xlsx	TC-004 is Fail; TC-005 is Blocked.

Required runtime behavior

1. Retrieve both relationships and preserve both source references.
2. Mark the linkage as conflicting rather than choosing one source.
3. Reduce confidence in the leakage conclusion.
4. Request verification of the authoritative test–defect relationship.
5. Do not block the entire analysis: continue with the evidence whose meaning is still valid.

Private evaluation truth

```
{
  "case_id": "DQ-001",
  "expected_finding": "conflicting_test_link",
  "sources": ["GH-ISSUE-107", "TC-004", "TC-005", "REQ-004"],
  "required_behavior": "surface_conflict_and_reduce_confidence"
}
```

Source note: traceability.csv, github_issues.csv and test_cases.xlsx.

1.6 Corrections before Google Cloud ingestion

Work item	Exact action	Done when
Metadata	Add source_system, source_uri, version, updated_at, authoritative, synthetic, quality_status and content_hash to every record.	Every claim can cite a stable evidence record.
QA identity mapping	Map QA-01/02/03 to canonical resource IDs; add the missing QA resource if required.	Tests and capacity analysis resolve to one resource model.
Issue lifecycle	Add synthetic created/updated timestamps, comments, labels, links and status transitions.	GitHub Issue and Project history can explain risk ageing and handoff.
Engineering lineage	Add requirement→file→symbol→commit→PR→test→build edges.	ENG.ai can traverse from mission to technical proof.
Code history	Create baseline, defect-introducing and fixed commits plus a branch and PR record.	RCA and change analysis use real diffs.
Build/test artifacts	Add failing reproduction, passing targeted tests and regression results.	No fix is accepted without executable proof.
Capacity realism	Add current allocation by sprint, effective capacity, active work, skills and backup owner.	Owner choice is evidence-backed.
Ground-truth isolation	Move risk_scenarios and expected technical cause outside runtime storage/indexing.	The agent discovers rather than retrieves answers.

1.7 Canonical Google-native data shape

Structured facts in BigQuery

```
features
```

```
requirements
test_cases
github_issues
resources
resource_allocations
sprints
milestones
traceability_edges
code_artifacts
commits
pull_requests
build_runs
```

Unstructured artifacts in Cloud Storage

- Project and architecture documents.
- Excel and CSV source snapshots.
- Source-code archive or repository export for ingestion.
- Build logs, test reports and patch artifacts.
- Versioned manifest for each published synthetic-project snapshot.

Runtime state in Firestore

```
investigations/{mission_id}
approvals/{approval_id}
handoffs/{handoff_id}
actions/{action_id}
sessions/{session_id}
evaluation_runs/{run_id}
```

Evidence envelope applied to every artifact

```
{
  "artifact_id": "TC-004",
  "artifact_type": "test_case",
  "source_system": "synthetic_qa",
  "source_uri": "qa://test-cases/TC-004",
  "version": "1.0",
  "updated_at": "...",
  "authoritative": true,
  "synthetic": true,
  "quality_status": "verified"
}
```

1.8 Expand the synthetic Git repository

The current code skeleton is sufficient to explain the product, but not enough to demonstrate a convincing code-level RCA, targeted fix and review flow. Add only the assets required by the golden scenario.

Addition	Purpose
tests/test_proximity_controller.py	Reproduce boundary oscillation and define expected unlock/lock behavior.
Baseline commit	Known good implementation state.
Regression-introducing commit	A small change that creates the GH-ISSUE-107 failure.
fix/gh-issue-107 branch	Isolated branch created after engineer approval.
Targeted patch	A deterministic correction such as hysteresis or stable-zone handling.
Failing and passing build records	Proof that the defect is reproduced and the fix is validated.
GH-PR-001 metadata	Title, description, linked GitHub Issue, reviewers, checks and merge status.
Code traceability edges	REQ-004 → file/symbol → commit → test → GH-ISSUE-107 → GH-PR-001.

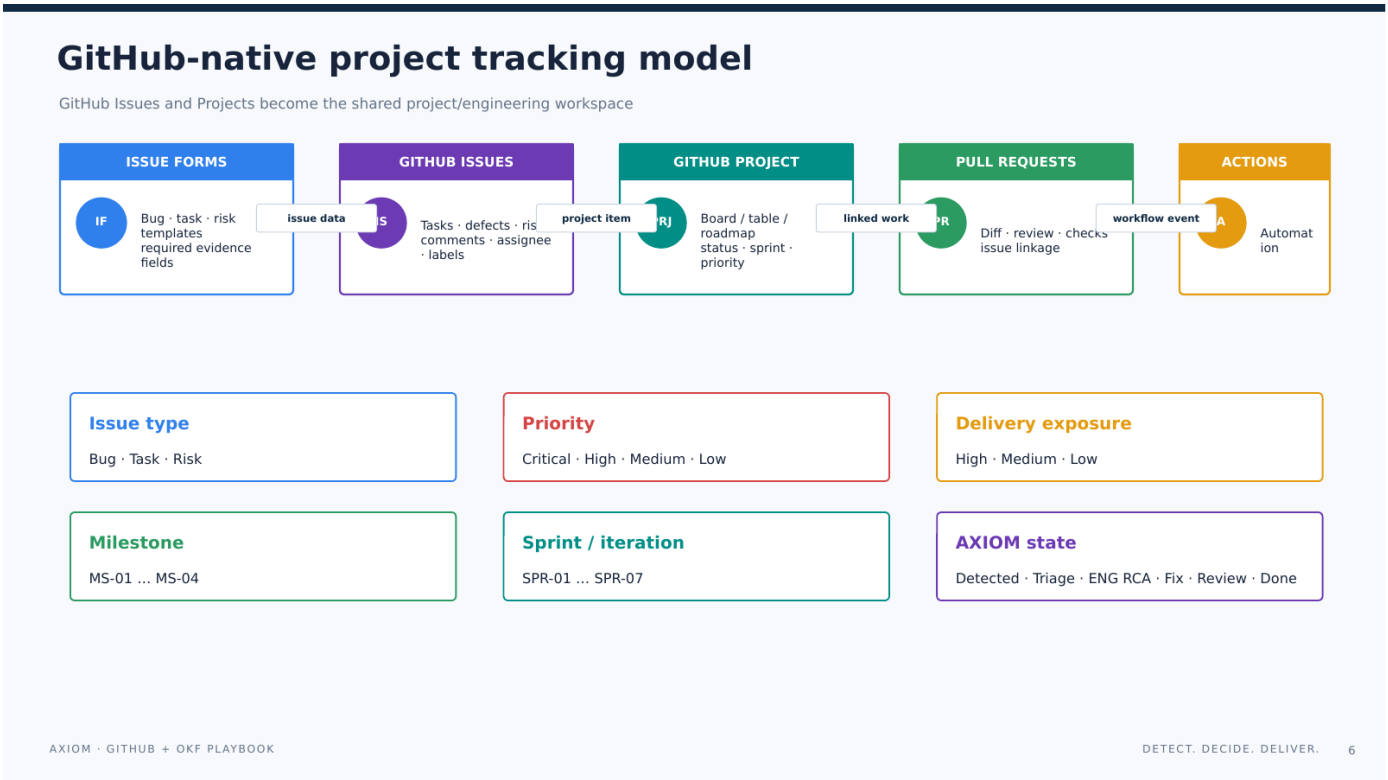
Recommended synthetic technical truth

The single-threshold boundary logic is unstable around the proximity boundary. The targeted correction adds a stable-zone/hysteresis rule and boundary regression tests.

This technical truth is a proposed synthetic scenario. It should be encoded in private evaluation truth and a reproducible failing commit, not exposed in runtime documents.

Source note: proximity_controller.py and test_digital_key.py; the proposed defect history and fix are new synthetic additions.

1.9 Configure the private GitHub Project



1.9 Configure the private GitHub Project: editable block-diagram view with named blocks and labeled flows.

Area	Configuration
Issue types	Epic, Story, Task, Bug and Risk.
Fields	Sprint, milestone, feature, requirement IDs, test IDs, delivery exposure, confidence, handoff mission ID, repository link and PR link.
Workflow	Open → Triage → PM Approved → Engineering Investigation → Fix Ready → Review → Done, plus Needs Evidence and Rejected.
PJM.ai write	Post preliminary analysis, evidence links, confidence, missing evidence and recommended owner after PM approval.
ENG.ai write	Post confirmed RCA, affected artifacts, patch summary, test evidence and PR link after engineer approval.
Audit	Record approver, action scope, timestamp, previous state and result.

GitHub reset state for every demonstration

- GH-ISSUE-107 status returned to Open.
- PJM.ai and ENG.ai comments removed or hidden from the reset snapshot.
- Original assignee and labels restored.
- Mission, approval and PR fields cleared.

- A scripted or documented reset path exists.

No GitHub Project status change, comment or reassignment before explicit PM approval.

1.10 Private ground truth and evaluation set

Package	Contents
Runtime corpus	Project facts, source documents, code, builds, GitHub and traceability.
Private truth	Expected risk chain, exact root cause, expected patch, expected tests, forbidden claims and expected tool order.
Evaluation cases	Golden case, conflict, missing evidence, rejected approval, unresolved RCA, failed patch validation, connector error and trajectory deviation.
Results	Final-response metrics, evidence metrics, tool-trajectory metrics, latency, failure and acceptance outcomes.

Minimum private truth record

```
{
  "case_id": "G-001",
  "query": "Resolve GH-ISSUE-107 end to end",
  "expected_project_evidence": ["REQ-004", "TC-004", "TC-005", "GH-ISSUE-107", "MS-02", "GH-ISSUE-119"],
  "expected_project_decision": "high_delivery_exposure",
  "expected_root_cause": "boundary_logic",
  "expected_patch": "stable_zone_or_hysteresis",
  "expected_tests": ["boundary_regression", "nominal_unlock", "passive_lock"],
  "forbidden_claims": ["PJM_confirms_code_root_cause"],
  "expected_tool_order": ["jira", "graph", "tests", "approval", "repo", "patch", "tests", "pr"]
}
```

Step 1 exit checklist

- All IDs, metadata and source links validate.
- QA owners map to the resource model.
- Code lineage and build history exist.
- The deliberate conflict is preserved and documented.
- Runtime and private truth are physically separated.
- The public package is scrubbed and versioned.

1.11 GitHub-native work tracking setup

Objective: use one GitHub repository, its Issues, one GitHub Project, branches, Actions and Pull Requests as the shared project-to-engineering system of record. The synthetic CSV remains an import source, not the runtime tracker.

A. Create the repository structure

6. Create a private implementation repository for the working prototype and a separate sanitized public repository for submission.
7. Create /.github/ISSUE_TEMPLATE for defect, task and risk issue forms.
8. Create /.github/pull_request_template.md for mandatory issue linkage, root-cause summary, test evidence and residual risk.
9. Create /knowledge/okf for generated Open Knowledge Format bundles.
10. Create /evaluation-private outside runtime ingestion.

B. Configure GitHub Issue Forms

- Defect form: summary, observed behavior, expected behavior, affected feature, milestone, sprint, requirement IDs, test IDs, severity evidence, reproduction evidence and logs.
- Task form: objective, acceptance criteria, dependencies, sprint, milestone and owner role.
- Risk form: trigger, affected commitment, exposure, likelihood, evidence IDs, mitigation owner and review date.
- Apply default labels such as type:defect, state:detected and axiom:triage-required.

C. Configure GitHub Project fields

- Status: Detected, Triage, PM Approval, Engineering Investigation, Fix Ready, Review, Done, Needs Evidence, Rejected.
- Issue Type: Defect, Task, Risk, Story.
- Priority: Critical, High, Medium, Low.
- Delivery Exposure: High, Medium, Low.
- Milestone: MS-01 to MS-04.
- Iteration: SPR-01 to SPR-07.
- AXIOM Mission ID, Confidence, Owner Role and Pull Request Link.

D. Migrate the synthetic tracker data

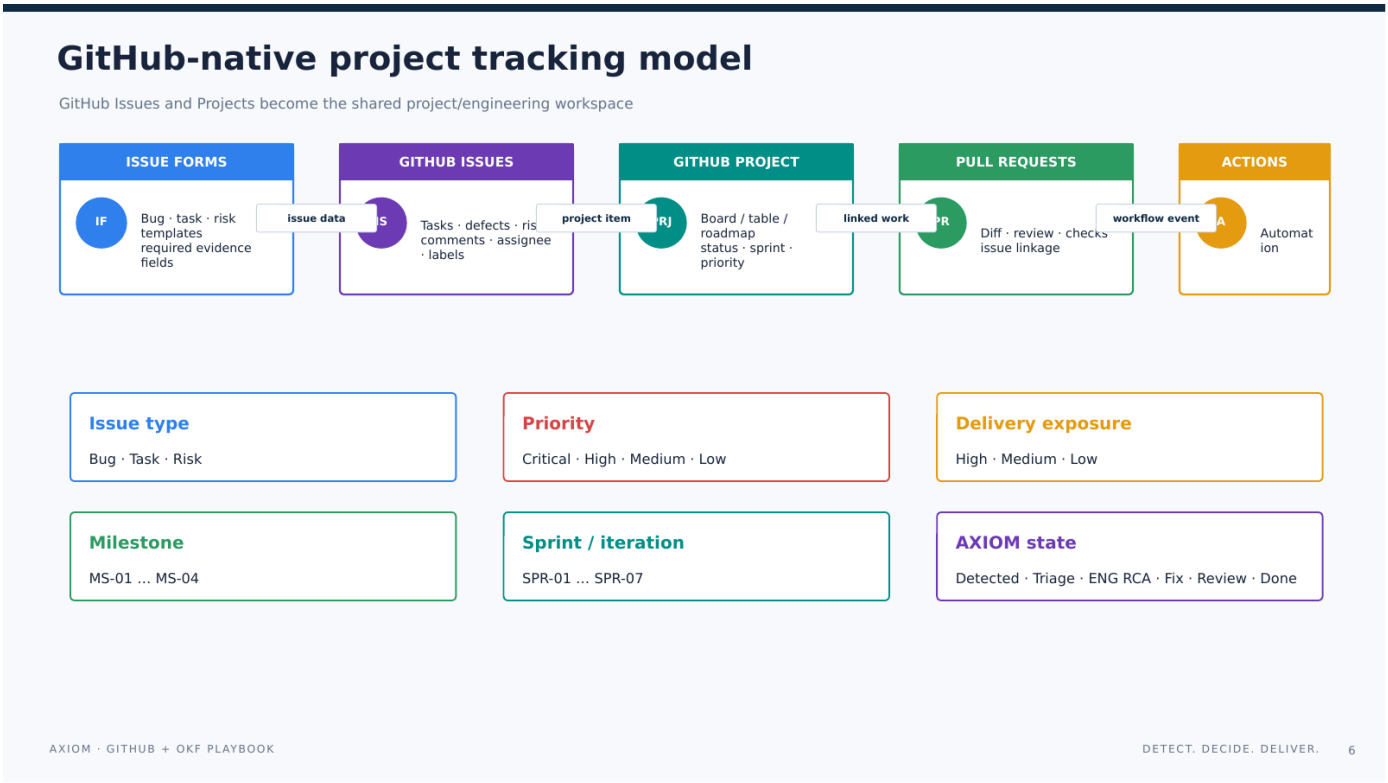
11. Map each synthetic tracker row to a GitHub Issue with a stable external_id field.
12. Preserve the original issue identifier in the issue body and OKF metadata; use the GitHub issue number as the runtime identifier.
13. Add issues to the GitHub Project and populate status, priority, iteration and milestone fields.
14. Create links from each Issue to requirement, test and OKF concept paths.
15. Validate that every defect used in the demo has an assignee, milestone, sprint/iteration and linked project evidence.

E. Define approved write operations

- PJM.ai may add a structured triage comment, assign the approved engineering owner and change the Project status only after PM approval.
- ENG.ai may create a branch, commit a patch, trigger GitHub Actions and create a Pull Request only after engineer approval.
- The connector must reject any write lacking approval_id, mission_id and expected previous state.
- Every write appends an audit record to Firestore and links the affected GitHub object.

GitHub tracking exit gate

Pass only when GH-ISSUE-107 can move from Detected to Engineering Investigation through PM approval, and then to Review through engineer approval, with the linked branch, checks and Pull Request visible from the same GitHub Project item.



1.12 Open Knowledge Format metadata layer

Decision: include OKF as a portable metadata and curated-context layer. Do not use OKF as the operational tracker, state database, graph engine or vector database.

A. Define the OKF bundle structure

- knowledge/okf/index.md
- knowledge/okf/project/e-key-2w.md
- knowledge/okf/features/<feature-id>.md
- knowledge/okf/requirements/<requirement-id>.md
- knowledge/okf/tests/<test-id>.md
- knowledge/okf/issues/<github-issue-number>.md
- knowledge/okf/code/<module-or-symbol>.md
- knowledge/okf/milestones/<milestone-id>.md

B. Generate YAML frontmatter

Every OKF concept should include type, title, description, resource, tags and timestamp, followed by AXIOM-specific fields such as artifact_id, project_id, version, source_uri, authoritative, synthetic and quality_status.

C. Generate body content

- Business or engineering meaning of the artifact.

- Important fields and their interpretation.
- Relationships expressed as Markdown links to other OKF concepts.
- External/source citations back to GitHub, the source snapshot or a build artifact.
- Known contradictions, missing evidence and confidence constraints when applicable.

D. Ingest and use OKF

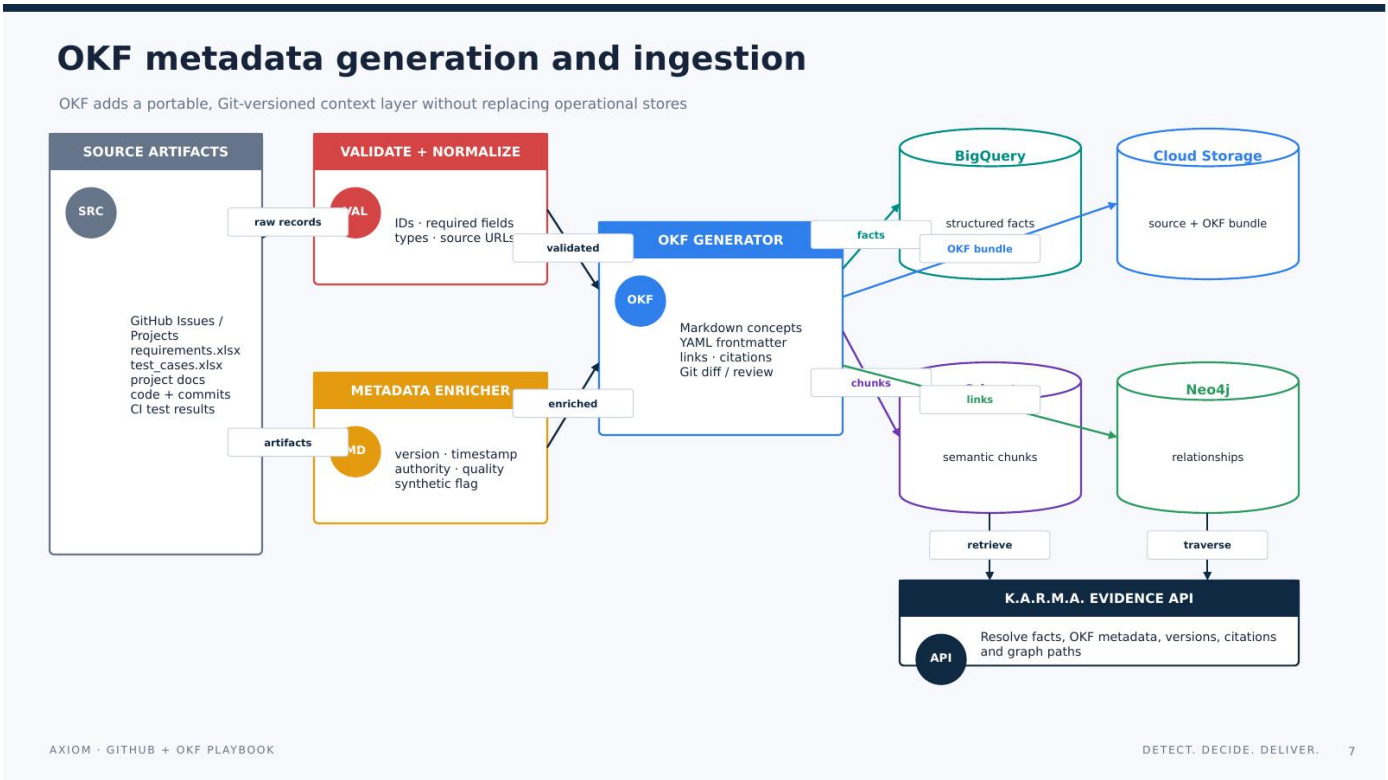
16. Commit generated bundles to the synthetic repository so changes are reviewed and diffable.
17. Store the released bundle in Cloud Storage under the project snapshot ID.
18. Index concept bodies in Qdrant for semantic retrieval.
19. Write concept relationships to Neo4j only when the relationship is validated.
20. Store structured operational fields in BigQuery; OKF references these facts rather than replacing them.
21. Expose OKF concept paths and citations through K.A.R.M.A. evidence responses.

E. Validate OKF

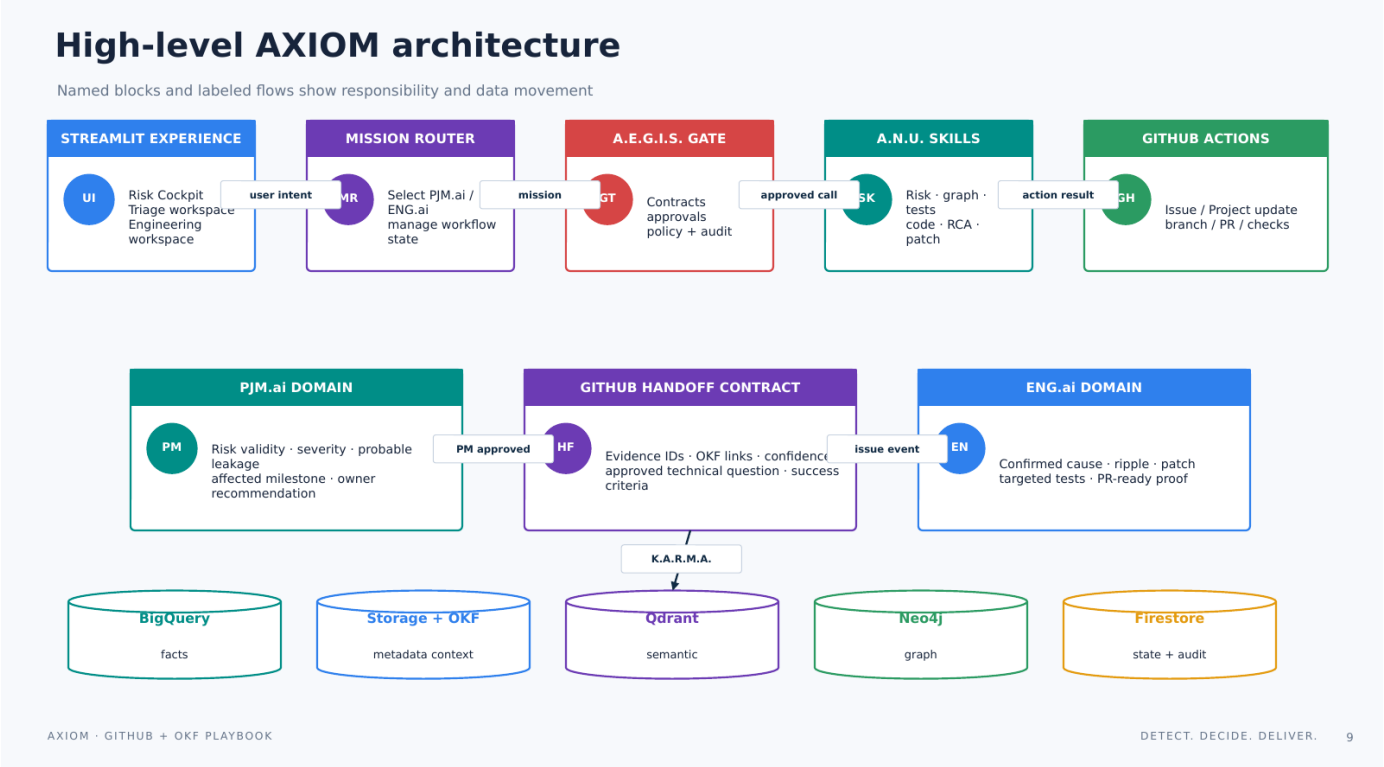
- All concept paths are unique.
- Required frontmatter exists and timestamps are valid.
- All internal links resolve.
- All source citations resolve.
- No evaluation-private expected answer is included.
- The same artifact ID and version resolve consistently across OKF, BigQuery, Qdrant and Neo4j.

OKF exit gate

Pass only when the golden scenario can retrieve project context from OKF, exact facts from BigQuery, semantic passages from Qdrant and dependency paths from Neo4j without duplicating or contradicting artifact identity.



Step 2 · AXIOM architecture transformation



Step 2 · AXIOM architecture transformation: editable block-diagram view with named blocks and labeled flows.

Objective: preserve the existing AXIOM concepts and domain boundaries while replacing the local and multi-provider runtime with a reliable, visibly Google-native vertical slice.

Target architecture concept: two bounded intelligence domains, one governed control plane, shared evidence and Google Cloud platform services

2.1 Transformation strategy

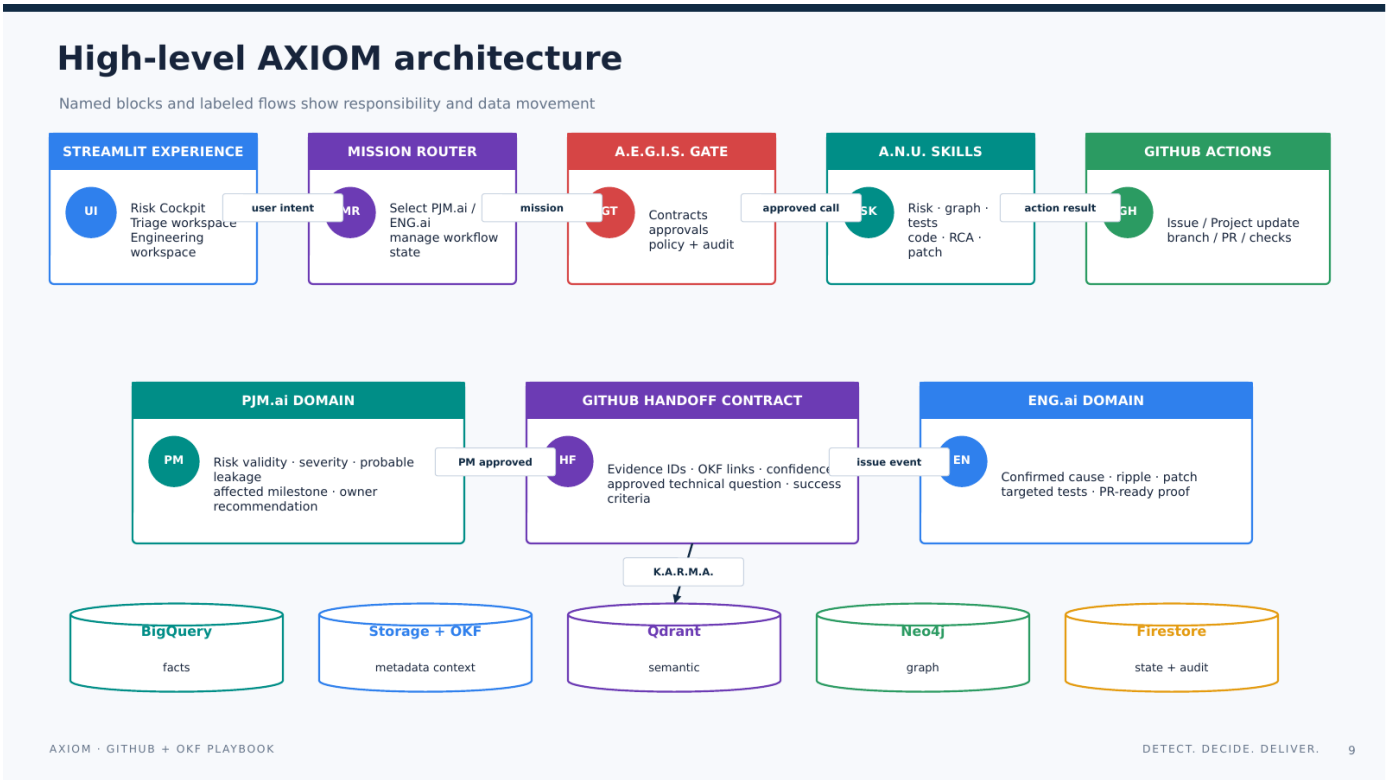
Cloud Run first. Gemini in both decision domains. ADK selectively. Preserve Qdrant and Neo4j behind bounded interfaces if they already work.

Current	Recommended target	Reason
Streamlit local	Streamlit container on Cloud Run	Minimal UI rewrite; fulfills cloud deployment requirement.

LangGraph + Claude Haiku	LangGraph or ADK-compatible state graph using Gemini on Vertex AI	Reduce model migration risk while making Gemini essential.
LangChain + GPT-4o-mini specialists	Bounded Gemini skills/tools; retain LangChain wrappers only if needed	Remove visible dependency on non-Google models in judging flow.
Langfuse	Cloud Logging/Trace and evaluation; Langfuse optional	Make Google-native observability visible; avoid unnecessary rewrite.
Qdrant	Retain behind a retrieval API or containerized service	No scoring value in replacing a working vector store.
Neo4j	Retain behind graph API; deploy privately or use a managed option	Blast-radius capability is a differentiator and should not be rebuilt late.
Local runtime	Independent Cloud Run services	Provides deployment, revisioning and isolation.

Source note: User-provided current stack. Google Cloud documentation for Cloud Run and ADK deployment.

2.2 High-level architecture



2.2 High-level architecture: editable block-diagram view with named blocks and labeled flows.

High-level AXIOM cloud architecture

USER EXPERIENCE
Streamlit Risk Cockpit | Triage Workspace | Engineering Workspace | Approval Panels

A.E.O.N. DOMAIN ORCHESTRATION
Mission Router → PJM.ai State Graph ↔ Governed Handoff ↔ ENG.ai State Graph

A.N.U. CAPABILITIES
Risk | Requirement Graph | Test | Resource | Code Search | RCA | Patch | Validation

A.E.G.I.S. CONTROL
Typed Contracts | HITL | Policy | Audit | Safe Failure

K.A.R.M.A. MEMORY
Evidence | Provenance | Versions | Graph | Contradictions | Confidence

O.L.I. EVALUATION
Traces | Final Answer | Tool Trajectory | Latency | Cost | Outcomes

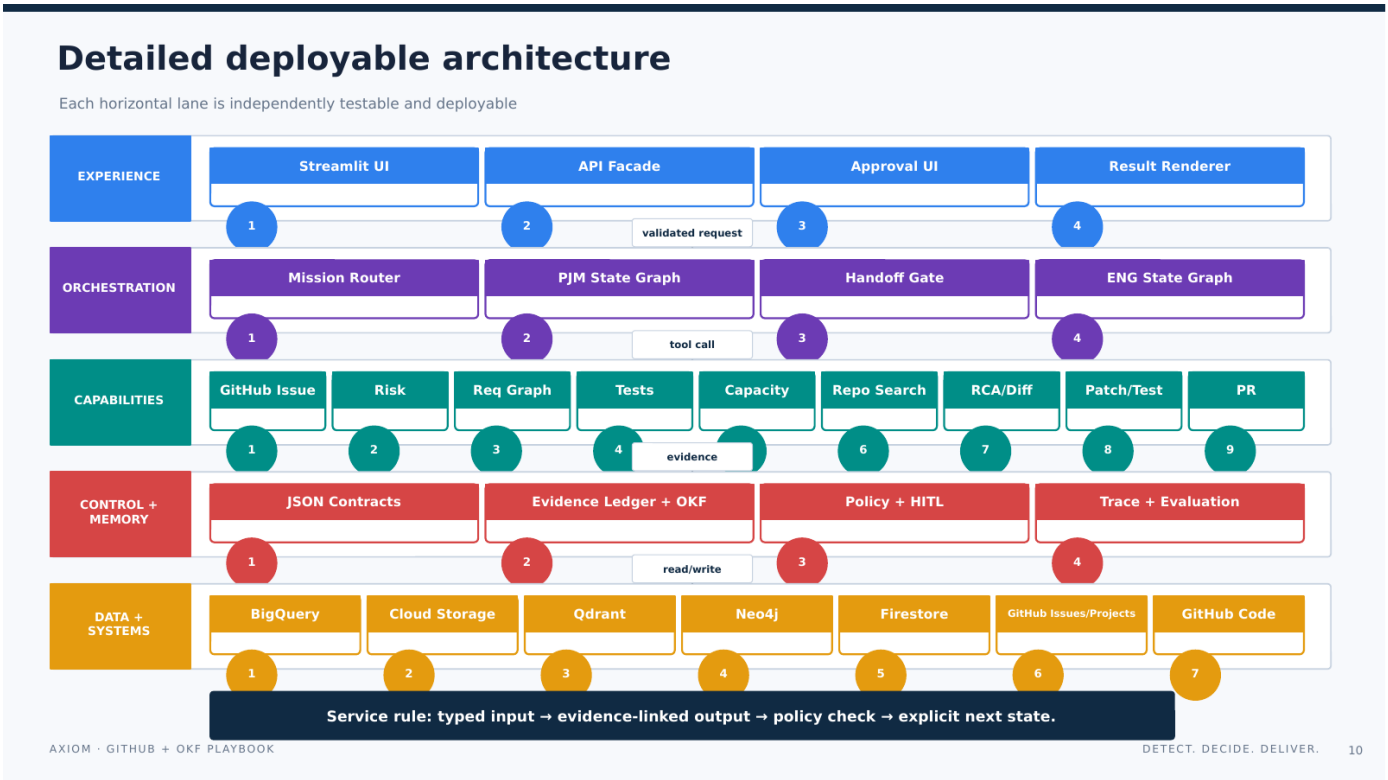
GOOGLE CLOUD PLATFORM
Cloud Run | Vertex AI Gemini | BigQuery | Cloud Storage | Firestore | Secret Manager | Monitoring | Build

Boundary principle

PJM.ai and ENG.ai share the evidence backbone and control plane but own different decisions. The architecture must not collapse them into one prompt or one unbounded agent.

Source note: PJM_ai_AXIOM_Doodle_Vector_Workshop_Deck 4.pptx and PJM_ai_Stabilization_AXIOM_Layer_Workflows_v4 2.pptx.

2.3 Detailed deployable architecture



2.3 Detailed deployable architecture: editable block-diagram view with named blocks and labeled flows.

Layer	Deployable components	Primary contracts
Experience	Streamlit UI; API gateway routes; role-aware pages.	Risk selection, triage render, approval requests, engineering render.
Orchestration	Mission router, PJM state graph, handoff gate, ENG state graph, response renderer.	Workflow state and typed node outputs.
Capabilities	GitHub Issue/Project tool, graph tool, test tool, capacity tool, repository search, diff tool, patch tool, test runner, PR tool.	Small tool-specific schemas.
Control	Schema validator, permissions, policy checks, approval service, audit writer, retry/failure handler.	A.E.G.I.S. action contract.
Memory	Evidence ledger, investigation state, handoff package, artifact version resolver.	K.A.R.M.A. evidence record.
Data	BigQuery, Cloud Storage, Firestore, Qdrant, Neo4j, GitHub Issues and Projects, Git repository.	Source and relationship model.
Platform	Cloud Run, Vertex AI Gemini, Secret Manager, Cloud Build, Artifact Registry, Monitoring/Trace.	Deployment and operations.

Recommended service decomposition

```
axiom-ui              # Streamlit
axiom-api             # authenticated backend façade
pjm-orchestrator      # project-delivery state graph
eng-orchestrator      # engineering RCA/fix state graph
evidence-service      # BigQuery/Storage/Qdrant retrieval
graph-service         # Neo4j dependency and ripple queries
action-service        # approved GitHub tracker/code mutations
evaluation-service    # case runner and score persistence
```

2.4 Freeze the PJM.ai ↔ ENG.ai decision boundary

PJM.ai owns	ENG.ai owns
Risk validity	Confirmed code/test root cause
Severity and urgency	Requirement–code–test trace
Probable leakage direction	Change and ripple impact
Affected delivery commitments	Corrective solution
Project-level blast radius	Targeted test definition and execution
Investigation ownership	Fix validation and residual technical risk
Replanning and escalation	PR-ready engineering evidence
Approved project-system actions	Approved engineering-system actions

PJM.ai must not

- Present a technical implementation conclusion as confirmed fact.
- Generate or apply a patch.
- Mark a defect fixed.
- Override the engineering owner's approval.

ENG.ai must not

- Change project priority or milestone status without PJM context.
- Act beyond the approved engineering mission.
- Merge a change without engineer approval and validation evidence.

Source note: PJM_ai_AXIOM_Architectural_Playbook_v1 1.docx and PJM_ai_Stabilization_AXIOM_Layer_Workflows_v4 2.pptx.

2.5 Orchestration decision

Option	Implementation	Use when	Recommendation
A · Retain LangGraph	Replace Claude/OpenAI calls with Gemini, containerize, formalize tools and JSON contracts.	Delivery risk is high and current graphs already work.	Recommended first.
B · Selective Google ADK	Use ADK for coordinator and/or bounded specialists while wrapping existing graph/vector services.	Team can absorb moderate framework change and wants stronger Google-native visibility.	Adopt selectively.
C · Full managed Agent Engine	Migrate orchestration, session and deployment patterns to the managed runtime.	Only after the end-to-end vertical slice is stable.	Do not make this a prerequisite.

Decision rule

Do not rewrite orchestration for branding. Win points by making Gemini essential, the deployment Google-native, and the evaluation measurable.

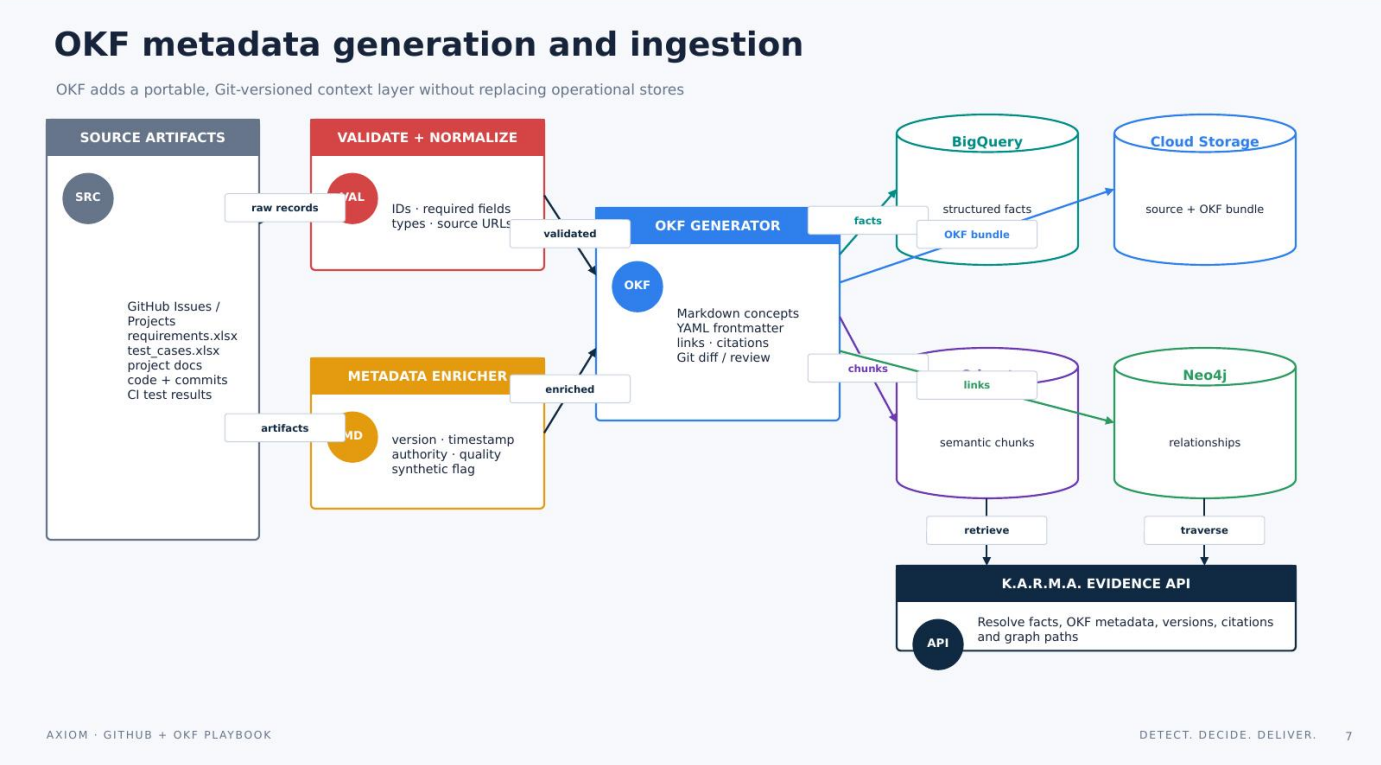
This recommendation is an implementation judgment based on the current stack and hackathon objective. Google ADK can be deployed on Cloud Run, but a full framework migration adds avoidable risk if the existing LangGraph flow is already functional.

2.6 Google Cloud service responsibilities

Service	Prototype responsibility
Cloud Run	Host Streamlit, API façade, PJM and ENG orchestrators, and bounded connector services.
Vertex AI Gemini	Semantic evidence synthesis, contradiction interpretation, bounded causal hypothesis, RCA explanation and patch explanation.
BigQuery	Store structured project facts and run deterministic joins, dates, counts and evaluation analytics.
Cloud Storage	Store source snapshots, documents, code/test artifacts and build reports.
Firestore	Persist sessions, approvals, handoffs, action records and evaluation run state.
Secret Manager	Store GitHub, Git and model credentials.
Cloud Build + Artifact Registry	Build, version and publish containers.
Cloud Monitoring / Trace	Capture latency, failures, tool sequence and operational metrics.

Optional Agent Search	Use only if it improves grounded retrieval beyond the current Qdrant evidence service.
Optional Agent Engine	Use after stability if managed runtime and trajectory tooling offer clear value.

2.7 Data ingestion and synchronization workflow



2.7 Data ingestion and synchronization workflow: editable block-diagram view with named blocks and labeled flows.

Data and workflow convergence visual

22. Validate file schemas and reject rows with missing primary IDs.
23. Normalize requirement, test, issue, resource, sprint, milestone and code identifiers.
24. Attach the mandatory evidence envelope and content hash.
25. Load structured facts into BigQuery.
26. Store source snapshots and artifacts in Cloud Storage.
27. Chunk defect narratives, requirements, tests and selected code for Qdrant retrieval.
28. Build Neo4j nodes and edges for project and engineering relationships.
29. Run integrity checks for orphan nodes, duplicate IDs, unresolved QA owners and invalid source URIs.
30. Publish an immutable project snapshot ID.
31. During the demo, write only approved GitHub tracker/code changes and append investigation/audit records to Firestore.

Integrity checks

- Every evidence ID resolves.

- Every graph edge has valid source and target.
- Every test owner maps to a resource or role record.
- Every artifact version has a content hash.
- No private truth file is present in the runtime index.

2.8 Retrieval and reasoning responsibilities

Mechanism	Must do	Must not do
BigQuery / Python	Dates, milestone proximity, allocation, thresholds, counts, status joins and deterministic scores.	Ask Gemini to calculate facts that can be derived exactly.
Neo4j	Requirement dependencies, project blast radius, code/test lineage and ripple queries.	Use a language model to invent graph paths.
Qdrant	Semantic retrieval of defects, requirements, tests and selected code/document chunks.	Treat similarity as proof.
Gemini	Interpret evidence meaning, reconcile or surface conflicts, formulate bounded hypotheses, explain impact and generate typed recommendations.	Create missing source facts or bypass contract validation.
Action tools	Apply only approved GitHub tracker or code mutations.	Execute from conversational intent without approval state.

Gemini interprets ambiguity. SQL, code and graph traversal enforce truth.

2.9 Workflow state and contracts

State	Required artifact	Transition gate
RISK_SELECTED	risk_id, project_id, project_snapshot_id	Risk and snapshot exist.
TRIAGE_COMPLETE	triage contract with evidence IDs	Schema valid; no unsupported claims.
PM_APPROVED	approval ID and action scope	Explicit Project Manager action.
HANDOFF_POSTED	engineering mission ID and GitHub Issue link	GitHub issue/Project write succeeds.
RCA_CONFIRMED	RCA contract and evidence	Confirmed cause is evidence-backed.
FIX_APPROVED	patch hash and reviewer	Explicit engineer action.

VALIDATED	targeted/regression test results and residual risk	Required tests pass or result is unresolved.
PR_READY	branch, PR ID and audit chain	All preceding gates complete.

Failure behavior

- No evidence: stop and show missing-evidence state.
- Invalid schema: retry once, then fail visibly with raw trace retained.
- No approval: block all writes.
- Conflicting evidence: preserve conflict, reduce confidence and request verification.
- Targeted test fails: do not mark fixed or prepare merge.
- Scope breach: return to the correct decision owner.

2.10 Governance and audit workflow

Control point	Checks
Before PM action	Risk exists; evidence references resolve; leakage remains preliminary; candidate owner is supported; action scope is visible; PM approval captured.
Before engineer action	Handoff approved; branch target and patch diff visible; tests and rollback path listed; engineer approval captured.
Before delivery proof	Patch applies; targeted tests pass; regression status is known; residual risk stated; PR record and audit chain exist.

Audit event

```
{
  "actor": "user_or_service",
  "action": "github_assign | git_apply_patch | create_pr",
  "artifact": "GH-ISSUE-107 | commit | GH-PR-001",
  "artifact_version": "...",
  "evidence_ids": ["..."],
  "approval_id": "...",
  "timestamp": "...",
  "result": "success | rejected | failed"
}
```

Trust must be visible in the UI, not hidden in backend logs.

2.11 O.L.I. observability and evaluation

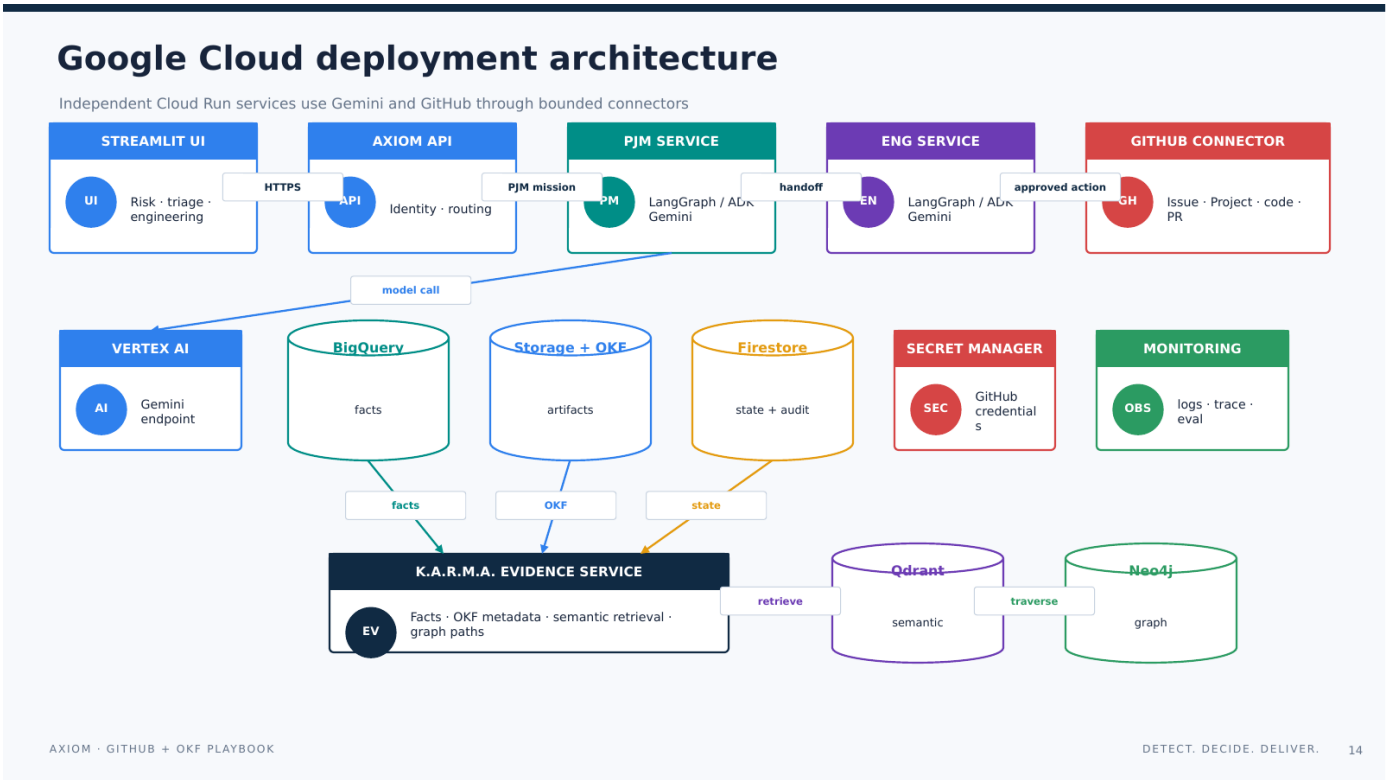
Dimension	Metrics
Final response	Risk correctness, RCA correctness, evidence citation precision/recall, unsupported-claim rate, patch explanation quality.
Tool trajectory	Called tools, order, completeness, unnecessary calls, retries and approval sequence.
Runtime	P50/P95 latency, service failures, model/tool exceptions, tokens and cloud cost.
Outcome	PM acceptance, engineer acceptance, time to actionable decision, patch applies, tests pass and PR created.

Recommended release gates

- 10 consecutive successful golden runs.
- 100% JSON-schema pass rate for triage, handoff, RCA and delivery contracts.
- Zero invalid source references.
- Zero GitHub tracker/code writes without approval.
- Targeted patch and tests pass in the golden case.
- Latencies remain inside the team's rehearsed demo envelope.

The exact latency threshold should be established from measured Cloud Run performance rather than guessed. The playbook therefore treats the threshold as a release parameter, not a sourced fact.

2.12 Cloud deployment sequence



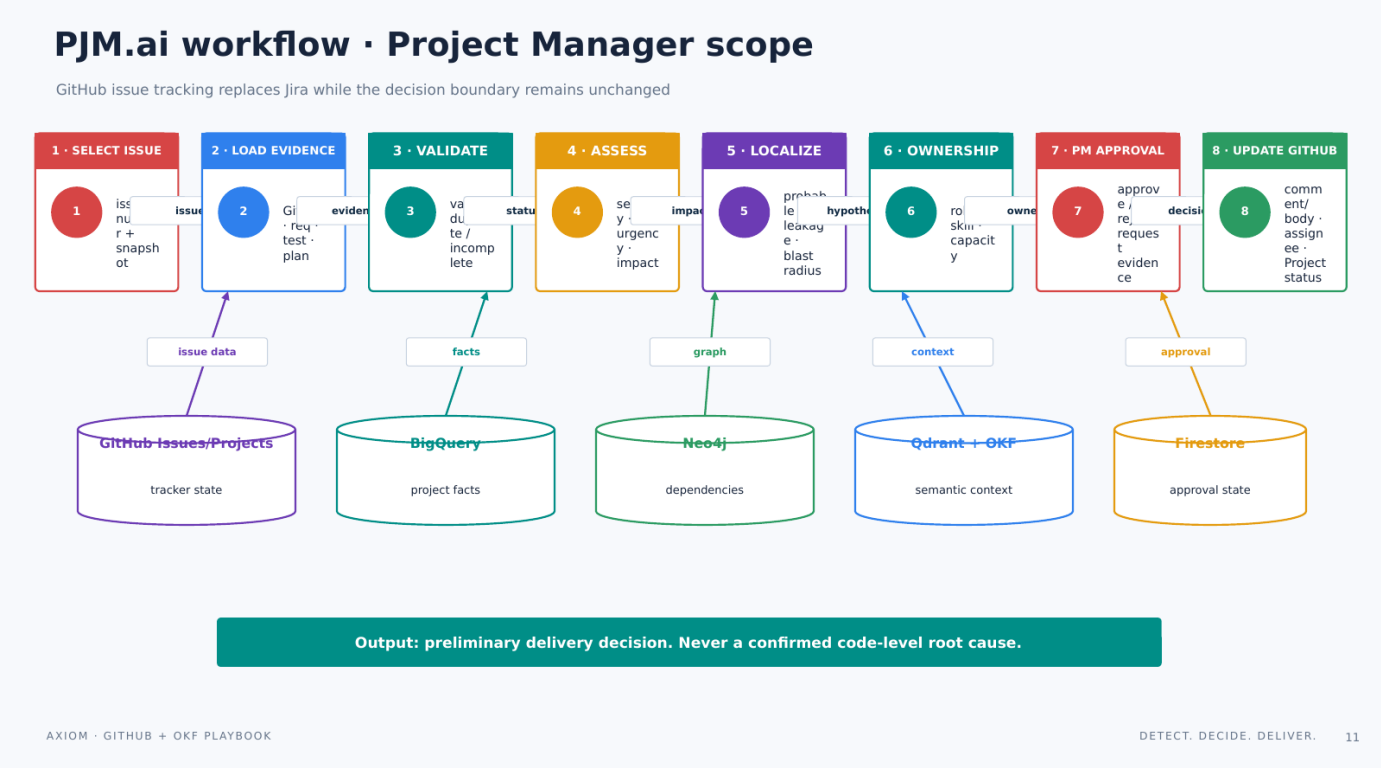
2.12 Cloud deployment sequence: editable block-diagram view with named blocks and labeled flows.

32. Create the Google Cloud project, budgets, APIs and least-privilege service accounts.
33. Containerize Streamlit, the API façade, PJM.ai orchestration, ENG.ai orchestration and supporting services.
34. Publish container images through Cloud Build and Artifact Registry.
35. Deploy independent Cloud Run services with health endpoints and revision labels.
36. Store GitHub Issues, Projects and code credentials in Secret Manager.
37. Load and validate the versioned synthetic project into BigQuery, Cloud Storage, Qdrant, Neo4j and Firestore.
38. Connect GitHub Issues, Projects and code through bounded connector tools.
39. Instrument logs, traces, mission IDs, evidence IDs, latency and errors.
40. Run the evaluation suite against the deployed endpoints.
41. Tag a release revision and document rollback to the last stable Cloud Run revision.

Step 2 exit checklist

- Streamlit cockpit is accessible from Cloud Run.
- Gemini is used in both PJM.ai and ENG.ai semantic reasoning.
- PJM.ai, ENG.ai and connectors are independently deployable.
- Typed contracts and approvals block invalid actions.
- GitHub read/write and Git read/branch/PR flows work against synthetic accounts.
- The deployed vertical slice is observable and reversible.

Step 3 · Golden demo execution



Step 3 · Golden demo execution: editable block-diagram view with named blocks and labeled flows.

Objective: prove one continuous risk-to-fix journey through two role-specific decision domains without losing evidence, governance or narrative continuity.

Six-scene demonstration arc from risk detection to verified delivery

3.1 Frozen input package

Input group	Frozen content
Risk trigger	GH-ISSUE-107: Boundary-zone unlock intermittently fails; Open; Critical; Sprint SPR-03; Milestone MS-02; assignee RES-04.
Project evidence	REQ-004, TC-004 Fail, TC-005 Blocked, REQ-006, TC-007 Fail, GH-ISSUE-108, GH-ISSUE-119, MS-02 At Risk.
Engineering evidence	proximity_controller.py, proximity regression tests, baseline commit, regression commit, failing build and expected requirement behavior.
User role	Project Manager selects the risk; engineer receives the approved mission and reviews the patch.
Private truth	Expected project chain, conflict behavior, exact synthetic root cause, targeted fix and expected test

	trajectory.

Source note: github_issues.csv, requirements.xlsx, test_cases.xlsx, resources.xlsx, traceability.csv, proximity_controller.py.

3.2 Stage 0 · Risk cockpit

User action

The Project Manager opens the Risk Cockpit and selects GH-ISSUE-107. The user does not start with an open-ended prompt.

Required cockpit content

- Risk status: High.
- Issue: GH-ISSUE-107.
- Affected milestone: MS-02 Passive Access Alpha.
- Why it surfaced: critical/open defect, failed or blocked validation, approaching milestone and capacity pressure.
- Linked evidence count and source types.
- Data-quality or missing-evidence badges.
- Investigate with AXIOM action.

Expected output

```
{
  "risk_id": "GH-ISSUE-107",
  "milestone_id": "MS-02",
  "risk_state": "high",
  "evidence_summary": ["requirement", "test", "defect", "capacity", "milestone"],
  "investigation_status": "not_started"
}
```

Pass condition

The cockpit explains why the item matters, but makes no RCA or fix claim.

3.3 Stage 1 · PJM.ai triage

Tool sequence

```
read_github_issue(GH-ISSUE-107)
```

```
→ query_requirement_graph(REQ-004)
→ read_test_evidence(TC-004, TC-005, TC-007)
→ calculate_delivery_exposure(MS-02, SPR-03)
→ query_capacity(RES-04, RES-05, RES-08, RES-09)
→ synthesize_triage_contract()
```

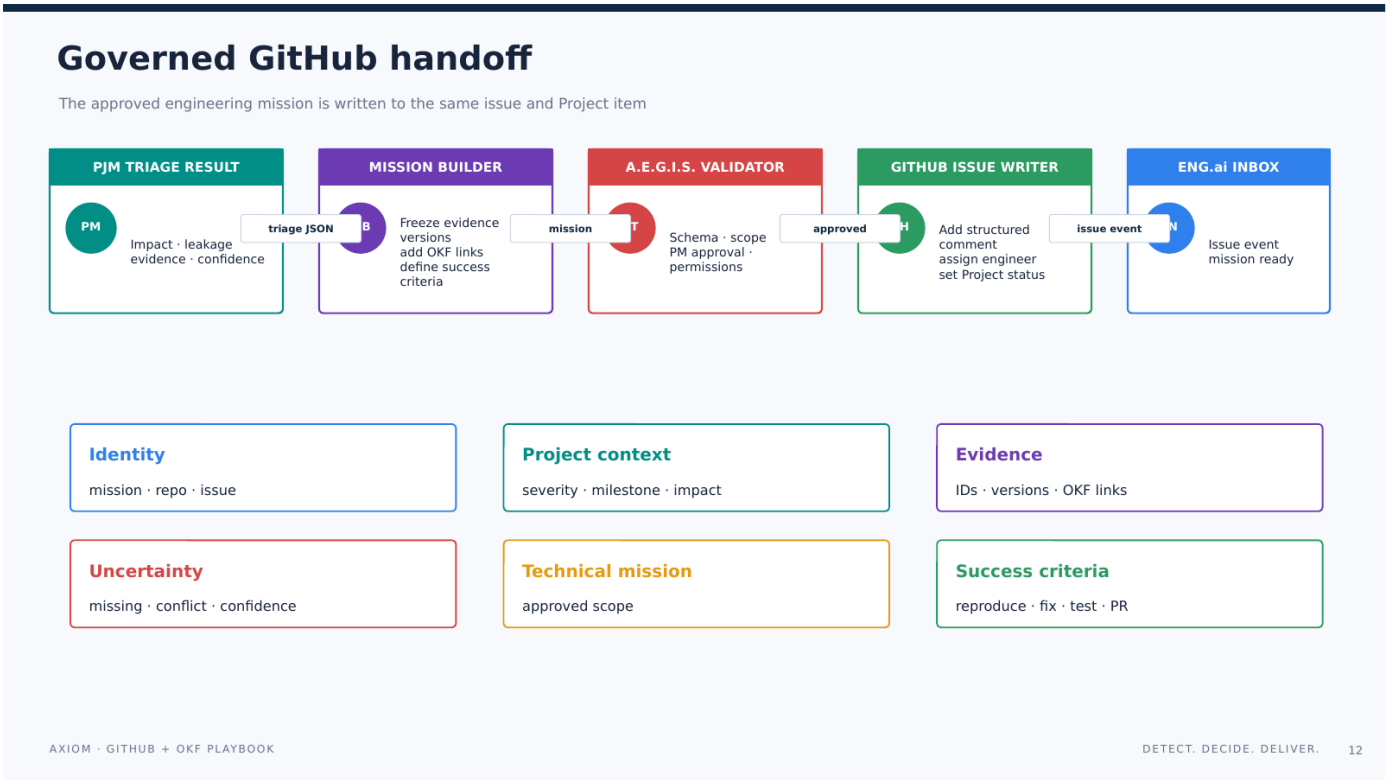
Expected triage output

```
{
  "validity": "confirmed",
  "severity": "critical",
  "delivery_exposure": "high",
  "affected_commitment": "MS-02",
  "probable_leakage": ["test_validation", "vehicle_software"],
  "conflicting_evidence": ["TC-004_vs_TC-005_link"],
  "missing_evidence": ["authoritative_GH-ISSUE-107_test_link"],
  "owner_recommendation": ["vehicle_software", "QA"],
  "confidence": "reduced",
  "approval_required": true
}
```

PJM.ai pass conditions

- All conclusions cite project evidence IDs.
- The conflict is surfaced and confidence is reduced.
- The result is labeled preliminary.
- No confirmed code root cause is claimed.
- An explicit PM approval request is shown.

3.4 PM approval and GitHub handoff



3.4 PM approval and GitHub handoff: editable block-diagram view with named blocks and labeled flows.

Handoff package

Field group	Required content
Identity	Mission ID, project, issue, requester and approved assignee.
Project decision context	Severity, urgency, delivery exposure and affected commitment.
Preliminary analysis	Probable leakage direction and alternative hypotheses.
Evidence	Artifact IDs, source versions, timestamps and links.
Uncertainty	Missing evidence, contradictions and confidence.
Technical mission	Approved technical question and permitted scope.
Success criteria	Reproduce, identify, patch, validate and prepare review evidence.

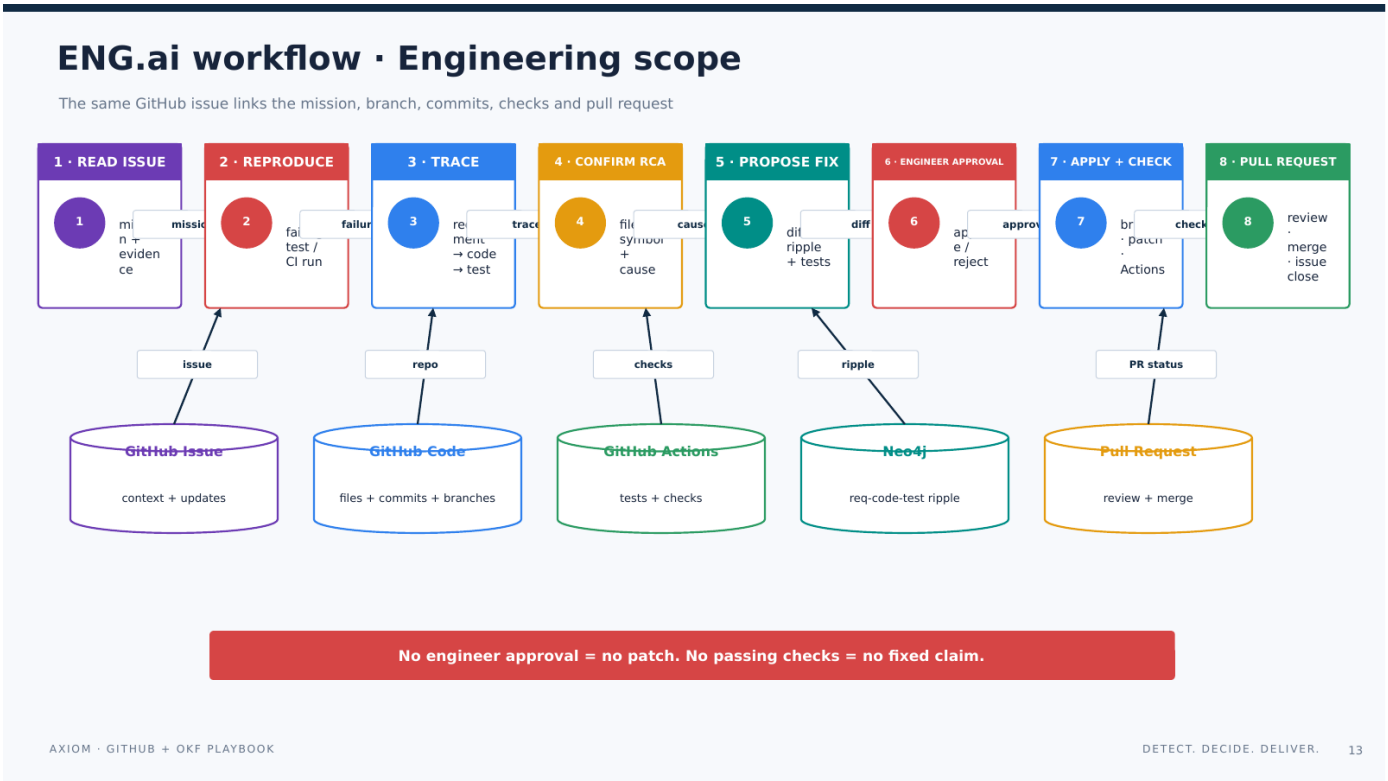
Required GitHub issue or Project mutation after approval

- Post the structured preliminary analysis.
- Attach or embed the mission package.

- Preserve evidence links.
- Assign the approved engineering owner.
- Transition to Engineering Investigation.
- Record the approval and previous state.

Pass when ENG.ai can start from the GitHub mission without asking the Project Manager to repeat context.

3.5 Stage 2 · ENG.ai RCA



3.5 Stage 2 · ENG.ai RCA: editable block-diagram view with named blocks and labeled flows.

Tool sequence

```
read_handoff(mission_id)
→ fetch_repository_snapshot(commit_sha)
→ reproduce_failure(test_boundary_zone)
→ trace_requirement_to_code(REQ-004)
→ inspect_symbol(proximity_controller.should_unlock)
→ compare_commits(baseline, regression)
→ assess_ripple(REQ-004, REQ-005, REQ-022)
→ synthesize_RCA_contract()
```

Expected RCA result


```
{
  "status": "confirmed",
  "root_cause": "unstable_single_threshold_boundary_logic",
  "affected_file": "src/vehicle/proximity_controller.py",
  "affected_symbol": "should_unlock",
  "requirement": "REQ-004",
  "reproduction_test": "test_boundary_zone_noise",
  "ripple_scope": ["passive_unlock", "passive_lock", "unknown_key_rejection"],
  "fix_strategy": "stable_zone_or_hysteresis",
  "engineer_approval_required": true
}
```


Pass conditions

- Failure is reproduced against the selected commit.
- Cause is connected to requirement, code and test evidence.
- Ripple impact is explicit.
- The patch has not yet been applied.
- Engineer approval is requested with a visible diff preview.

3.6 Targeted fix, engineer approval and verification

42. ENG.ai proposes a minimal diff and explains why the change addresses the confirmed cause.
43. The engineer reviews affected files, ripple impact, target branch, tests and rollback path.
44. The engineer approves or rejects the action.
45. On approval, the action service creates fix/gh-issue-107 and applies the patch.
46. The new boundary regression test is run and must fail against the original state and pass after the patch.
47. Nominal unlock, passive lock and unknown-key rejection regressions are run.
48. ENG.ai records results, residual risk and patch hash.
49. A pull request is created and linked to the GitHub Issue.
50. Merge is performed only according to the chosen demo policy; a PR-ready state is sufficient if live merge is considered risky.

Expected delivery result


```
{
  "branch": "fix/gh-issue-107",
  "patch_applied": true,
  "targeted_tests": "pass",
  "regression_tests": "pass",
  "residual_risk": "stated",
  "pr_id": "GH-PR-001",
  "github_issue_updated": true,
  "audit_complete": true
}
```

No passing tests, no fixed claim. No engineer approval, no patch.

3.7 Test matrix

ID	Case	Expected behavior
G-01	Golden GH-ISSUE-107 journey	Correct risk chain, PM handoff, confirmed RCA, approved patch, passing tests and PR proof.
G-02	Repeat same risk	Stable evidence selection and contract output.
DQ-01	Conflicting test link	Conflict surfaced; confidence reduced; verification requested.
ME-01	TC-005 removed	Missing-evidence state; analysis remains bounded.
AP-01	PM rejects approval	No GitHub Issue comment, assignment or transition.
RC-01	Repository cause removed	ENG.ai returns unresolved rather than fabricating.
FX-01	Engineer rejects patch	No branch, commit or PR.
TV-01	Targeted test fails after patch	No fixed claim; mission remains open.
SEC-01	Invalid GitHub tracker/code secret	Safe connector error with no leakage.
TR-01	Unexpected tool order	Trajectory evaluation fails.

Acceptance metrics

- Risk classification and evidence precision/recall.
- Contradiction and missing-evidence detection.
- RCA correctness.
- Patch validity and test result.
- Schema pass rate.
- Action-without-approval count must remain zero.
- Tool trajectory and latency.

3.8 Three-minute demo storyboard

Time	Presenter action	Visible system proof
0:00–0:20	Open Risk Cockpit and select GH-ISSUE-107.	Risk, milestone and evidence sources are visible.

0:20–1:05	Run PJM.ai triage.	Evidence map, severity, leakage, conflict, confidence and owner appear.
1:05–1:25	Approve handoff and open GitHub.	Structured report is posted, owner assigned and mission preserved.
1:25–2:10	Start ENG.ai from GitHub mission.	Failure reproduction, req–code–test trace and confirmed cause appear.
2:10–2:40	Review and approve patch.	Diff, targeted test and regression results appear.
2:40–3:00	Show PR/audit and close.	PR link, GitHub Issue and Project update, quality metric and final promise are visible.

One risk. One governed handoff. One verified change. One measurable outcome.

Presenter closing line

“AXIOM did not simply answer a question. It connected fragmented project evidence, governed the delivery decision, established engineering truth and delivered a verified change.”

3.9 Demo operator runbook

Before every run

- Reset GitHub Issue and Project status, assignee, comments, labels and mission fields.
- Delete or close the previous Git branch and PR, or reset to a clean repository snapshot.
- Load the frozen project snapshot ID.
- Warm all Cloud Run services and confirm health checks.
- Verify Vertex AI, GitHub, Git, Qdrant and Neo4j connectivity.
- Run a quick read-only smoke test.

During the run

- Follow the storyboard exactly.
- Show citations and confidence before approval.
- Pause visibly at PM and engineer approval gates.
- Show the GitHub handoff before invoking ENG.ai.
- Show the failing test before accepting the fix.
- Show the test and PR proof after the fix.

After the run

- Open the audit timeline.
- Open the before/after targeted test evidence.
- Show the evaluation result and latency.
- Close with the affected milestone and resolved engineering risk.

Fallback

Keep a deterministic recording of the exact same deployed flow for submission and contingency. Do not replace the working product with a click-through mock-up.

3.10 Winning scorecard

Judging dimension	Visible proof in the demo
Technical merit and GenAI implementation · 40%	Deployed Cloud Run application; Gemini used meaningfully in triage and RCA; bounded tool calls; typed contracts; working GitHub tracker/code actions; evaluation trace.
Problem alignment and impact · 25%	A real project-delivery risk is connected to an affected milestone and driven to a governed resolution.
Innovation and creativity · 25%	Two decision domains; causal evidence; contradiction handling; confidence; project-to-engineering continuity; verified change.
User experience and solution design · 10%	Role-aware cockpit and engineering workspace; visual evidence; explicit approvals; no context loss through GitHub.

10/10 readiness definition

Deployed · Measured · Repeatable · Governed · Visually obvious · Completed inside three minutes

3.11 Versioned workflow contracts

These contracts must be implemented as versioned JSON schemas and used by the UI, orchestrators, GitHub connector, audit service and evaluation suite.

RiskSelection

repo, issue_number, project_snapshot_id, user_role

EvidenceQuery

artifact_ids, graph_depth, freshness_requirement, allowed_sources

TriageDecision

risk_validity, severity, delivery_exposure, probable_leakage, affected_commitment, evidence, conflicts, missing_evidence, owner_recommendation, confidence

ApprovalDecision

approval_id, mission_id, actor, scope, decision, timestamp

EngineeringMission

mission_id, repository, issue_number, project_context, evidence_versions, OKF_links, uncertainty, approved_question, success_criteria

RCAResult

status, confirmed_cause, affected_files, affected_symbols, reproduction, ripple_scope, correction_options, targeted_tests, confidence

PatchAction

branch, base_commit, patch_hash, files, tests, approval_id

ValidationEvidence

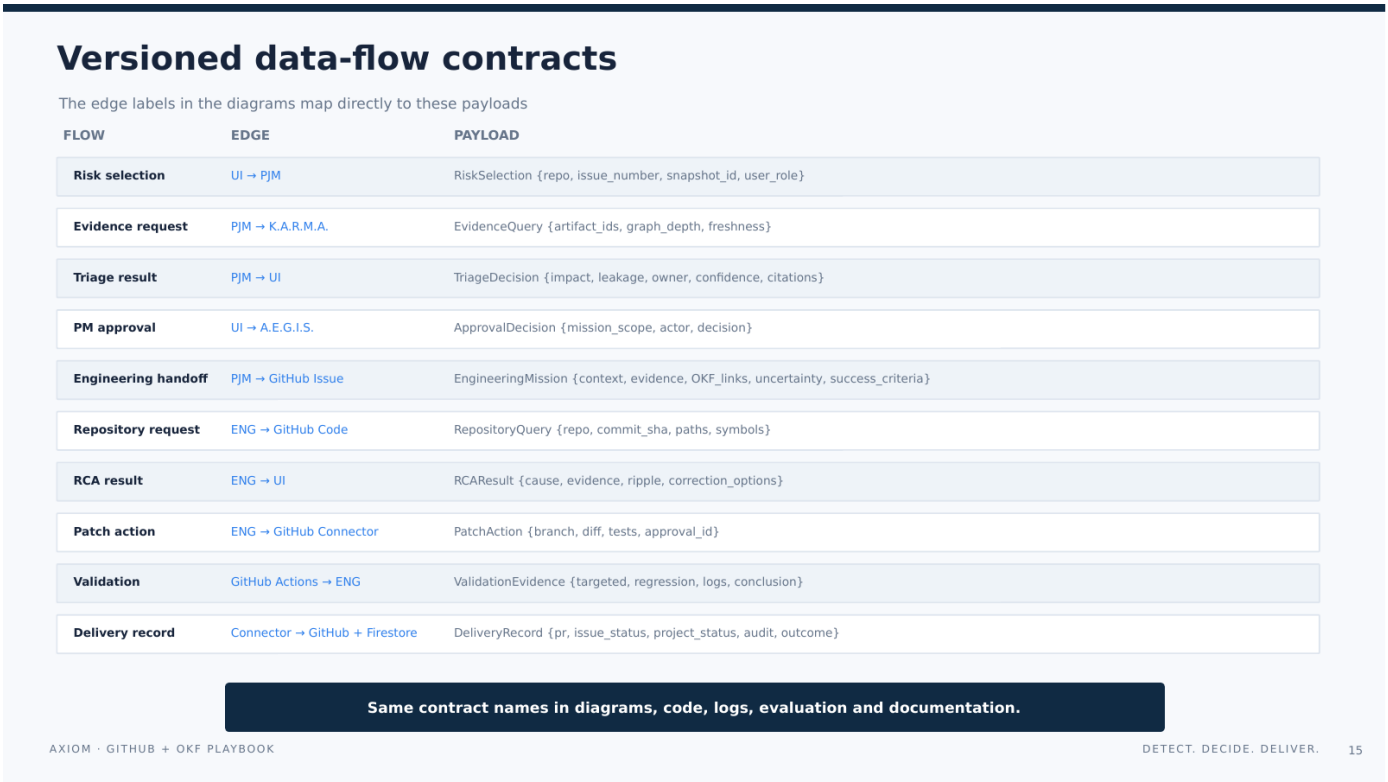
workflow_run_id, targeted_results, regression_results, logs, conclusion

DeliveryRecord

pull_request, issue_status, project_status, audit_events, residual_risk, outcome

Contract gate

Reject the transition when required fields are absent, source IDs do not resolve, the expected previous state does not match, or the required approval is absent.



Final submission gate

☐ Public synthetic repository contains no Bosch or customer material.

☐ All documents and Git history are scrubbed.

☐ Runtime cannot retrieve private ground truth.

☐ Google Cloud endpoint is stable and documented.

☐ Gemini contribution is visible in PJM.ai and ENG.ai.

☐ GitHub Issues, Projects and code use synthetic accounts.

☐ PJM.ai never overclaims technical truth.

☐ ENG.ai shows evidence-linked RCA.

☐ Both approvals block writes.

☐ Targeted test fails before and passes after.

☐ PR or merge evidence is visible.

☐ Ten consecutive golden runs succeed.

☐ The three-minute video tells one continuous story.

☐ README, architecture, deck and UI use the same terms.

READY = one risk fully resolved with evidence, control and proof.

Immediate next execution sequence

51. Freeze G-001: GH-ISSUE-107 Passive Access Alpha.
52. Create the missing code history, failing test, targeted fix and PR metadata.
53. Normalize and version the synthetic data package.
54. Finalize triage, handoff, RCA and delivery JSON schemas.
55. Deploy the current workflow to Cloud Run with Gemini.
56. Connect GitHub Issues, Projects and code synthetic accounts.
57. Run the test matrix and fix every non-deterministic outcome.
58. Record the three-minute golden demonstration.

Source register

Enterprise and user-provided sources used in this playbook:

- project_manifest.json
- project_description.md
- project_plan.md
- risk_scenarios.md
- README.md
- github_issues.csv
- requirements.xlsx
- test_cases.xlsx
- resources.xlsx
- traceability.csv

- architecture.md
- key_service.py
- digital_key_service.py
- proximity_controller.py
- ride_authorization.py
- test_digital_key.py
- PJM_ai_AXIOM_Architectural_Playbook_v1 1.docx
- PJM_ai_Stabilization_AXIOM_Layer_Workflows_v4 2.pptx
- PJM_ai_AXIOM_Doodle_Vector_Workshop_Deck 4.pptx
- PJM_ai_Stabilization_Team_Workshop_Deck_v2_Labeled_AEGIS 2.pptx

External references: Google Cloud ADK deployment to Cloud Run, Google Cloud single-agent architecture, Google Cloud agent evaluation documentation, and AI Builder Cup judging/submission requirements.

External reference additions: GitHub Issues and Projects documentation; GitHub Issue Forms syntax; Google Open Knowledge Format specification and Google Cloud introduction to OKF.